



GLOSSÁRIO TÉCNICO • SPRINT 2 • CAPÍTULO 2

Working with Text Data

De texto bruto a Token IDs, embeddings e posições:
o pré-processamento de dados de um LLM

TOKENIZAÇÃO • VOCABULÁRIO • TOKEN IDS • EMBEDDINGS • DATALOADER

Sumário

Introdução	1
1 Tokenização e tokens	1
2 Vocabulário e Token IDs	2
3 Sequências de treinamento	4
4 Embeddings e posição	6

Introdução

Este glossário registra os conceitos introduzidos no Capítulo 2 — *Working with Text Data* — do livro *Build a Large Language Model (From Scratch)* (Sebastian Raschka, Manning, 2025), referência principal do Projeto LLM. O objetivo não é traduzir os termos da língua inglesa, mas registrar de forma sintética seu significado e sua função no pipeline de pré-processamento de dados de um modelo de linguagem. O glossário é cumulativo em relação ao produzido na Sprint 1 (ver `LLM_Glossario_IA.tex`) e deverá ser reutilizado nas próximas Sprints.

1 Tokenização e tokens

Tokenization

Tokenização

Definição: Processo de dividir uma sequência contínua de texto em unidades menores e discretas — os tokens — que podem ser palavras, partes de palavras, sinais de pontuação ou até caracteres/bytes individuais, dependendo do esquema utilizado.

Função no modelo: É a primeira etapa do pipeline: transforma um objeto que a rede não sabe processar (uma *string*) em uma sequência de unidades discretas que podem, na etapa seguinte, ser mapeadas para números.

Relação com outros conceitos: Precede a construção do *Vocabulário* e a conversão para *Token ID*. No GPT, a tokenização é feita por *Byte Pair Encoding (BPE)*, não pela versão simples baseada em regex.

Exemplo: "Hello, do you like tea?" → ["Hello", ",", "do", "you", "like", "tea", "?"] (tokenização por regex, separando palavras e pontuação).

Token

Token

Definição: Unidade mínima produzida pela tokenização. Pode corresponder a uma palavra inteira, a um pedaço de palavra (subword) ou a um sinal de pontuação — nunca a um conceito semântico completo por si só.

Função no modelo: É a unidade de trabalho de todo o pipeline: o texto deixa de ser manipulado como caracteres soltos e passa a ser manipulado como uma sequência de tokens, cada um posteriormente associado a um *Token ID* e a um vetor de *Embedding* (*representação vetorial*).

Relação com outros conceitos: Um token não carrega significado por si — seu “significado” computacional só surge depois, quando seu *Token ID* é convertido em vetor pela camada de embedding.

Exemplo: Na frase “the sunlit terraces”, o BPE do GPT-2 produz os tokens ["the", "sun", "lit", "terr", "aces"] — note que “sunlit” e “terraces” são quebrados em subpalavras, pois não aparecem inteiras no vocabulário do tokenizador.

Corpus

Corpus (de texto)

Definição: Conjunto de texto bruto utilizado como fonte de dados — para a construção do vocabulário, para a extração de sequências de treinamento, ou para o pré-treinamento do modelo.

Função no modelo: É a matéria-prima de toda a Sprint: sem um corpus não há tokens, não há vocabulário e não há sequências de treinamento.

Relação com outros conceitos: Alimenta diretamente a *Tokenization*; seu tamanho (em tokens) determina quantas amostras de *Sequência de treinamento* (*par entrada/alvo*) podem ser extraídas

para um dado tamanho de contexto.

Exemplo: Neste projeto, utiliza-se o texto “*The Verdict*” (Edith Wharton, domínio público), o mesmo corpus de exemplo do capítulo, com aproximadamente 20 479 caracteres e 5 145 tokens BPE.

2 Vocabulário e Token IDs

Vocabulary

Vocabulário

Definição: Conjunto de todos os tokens distintos que um tokenizador é capaz de reconhecer, organizado como um dicionário que associa cada token a um número inteiro único (seu *Token ID*).

Função no modelo: Define o espaço discreto sobre o qual o modelo opera: todo token de entrada precisa existir no vocabulário (ou ser tratado por um token especial) para ser convertido em ID, e todo token gerado pelo modelo na saída é, na verdade, um índice desse mesmo vocabulário.

Relação com outros conceitos: Construído a partir dos tokens únicos de um *Corpus*; define o tamanho da tabela de *Token embedding* (`vocab_size` linhas) e do vetor de *logits* produzido na camada final do modelo.

Exemplo: No corpus *The Verdict*, o vocabulário construído a partir dos tokens únicos possui 1 130 entradas; somando os dois tokens especiais (`<|unk|>` e `<|endoftext|>`), chega a 1 132. Já o vocabulário BPE do GPT-2, fixo e independente do corpus, possui 50 257 entradas.

Token ID

Token ID (identificador do token)

Definição: Número inteiro que representa um token dentro do vocabulário — o índice da linha correspondente a esse token na tabela de vocabulário.

Função no modelo: É a única representação numérica bruta do texto que a rede manipula antes da camada de embedding. Serve como índice de consulta (*lookup*) na tabela de *Token embedding*.

Relação com outros conceitos: Obtido a partir de um *Token* via o *Vocabulary*; convertido de volta em token pela operação inversa (*decoding*); não deve ser confundido com uma representação semântica — dois IDs numericamente próximos (ex.: 5 e 6) não implicam tokens semanticamente relacionados.

Exemplo: “Hello” → Token ID 15496 no vocabulário BPE do GPT-2 (`tiktoken`, `encoding="gpt2"`).

Special tokens

Tokens especiais

Definição: Tokens artificiais, sem correspondência direta no texto original, adicionados ao vocabulário para tratar casos que a tokenização normal não resolve. Os dois discutidos no capítulo são `<|unk|>` (*unknown*, para palavras fora do vocabulário) e `<|endoftext|>` (para marcar a fronteira entre documentos concatenados).

Função no modelo: `<|unk|>` evita que o *Tokenization* falhe diante de uma palavra nunca vista; `<|endoftext|>` evita que o modelo aprenda associações espúrias entre o fim de um texto e o início de outro quando vários documentos são concatenados em um único fluxo de treinamento.

Relação com outros conceitos: São entradas adicionais no *Vocabulary*, recebendo seus próprios *Token ID*; o tokenizador *Byte Pair Encoding* (BPE) do GPT, por lidar com qualquer string via subpalavras, normalmente dispensa o `<|unk|>` — mas mantém o `<|endoftext|>`.

Exemplo: “Hello, do you like tea?” contra um vocabulário que não conhece “Hello” produz [`<|unk|>`, “,”, “do”, “you”, “like”, “tea”, “?”].

Byte Pair Encoding (BPE)

Codificação por Pares de Bytes

Definição: Algoritmo de tokenização em nível de subpalavra que constrói seu vocabulário mesclando iterativamente os pares de caracteres (ou bytes) mais frequentes em um corpus de treinamento, até atingir um tamanho de vocabulário alvo.

Função no modelo: É o tokenizador efetivamente usado pelo GPT. Por operar em subpalavras e, no limite, em bytes individuais, consegue representar qualquer string de entrada — inclusive palavras inventadas ou fora do vocabulário — sem nunca precisar de um token `<|unk|>`.

Relação com outros conceitos: Substitui, na prática, o *Tokenization + Vocabulary* caseiros descritos nas seções iniciais do capítulo; implementado neste projeto via a biblioteca `tiktoken` (`encoding="gpt2"`), com vocabulário fixo de 50 257 tokens.

Exemplo: A palavra inventada `"Akwirw ier"` é quebrada pelo BPE em `["Ak", "w", "ir", "w", " , "ier"]` — seis subpalavras que, recombinações, reconstroem exatamente a string original.

3 Sequências de treinamento

Sliding window

Janela deslizante

Definição: Técnica de amostragem que percorre a sequência de Token IDs do corpus com uma janela de tamanho fixo (`max_length`), avançando um número fixo de posições (`stride`) a cada passo, para extrair múltiplas subsequências de treinamento a partir de um único corpus contínuo.

Função no modelo: É o mecanismo que transforma um corpus (uma longa sequência de tokens) em um conjunto de exemplos de treinamento de tamanho fixo, prontos para compor lotes (*batches*).

Relação com outros conceitos: Produz os pares de *Sequência de treinamento* (*par entrada/alvo*); o `stride` controla a sobreposição entre janelas consecutivas e, junto com o *Context length* (*tamanho do contexto*), determina a quantidade total de amostras geradas.

Exemplo: Com `max_length=4` e `stride=4` (sem sobreposição), o corpus *The Verdict* (5 145 tokens BPE) produz 1 286 amostras; com `stride=1` (máxima sobreposição), produziria cerca de 5 141 amostras.

Input-target pair

Par entrada/alvo (sequência de treinamento)

Definição: Par de sequências de mesmo tamanho em que o alvo (*target*) é exatamente a sequência de entrada (*input*) deslocada em uma posição para a direita.

Função no modelo: Implementa, de forma totalmente automática, a tarefa de auto-supervisão do LLM: “prever o próximo token”. Não é necessária nenhuma rotulação manual — o próprio texto fornece o rótulo de cada posição.

Relação com outros conceitos: Gerado pela *Sliding window* sobre os *Token ID* do corpus; consumido pelo *DataLoader*, que os agrupa em lotes (*Batch* (*lote*)) para alimentar o modelo durante o treinamento.

Exemplo: Para a sequência de IDs [40, 367, 2885, 1464]: entrada = [40, 367, 2885], alvo = [367, 2885, 1464]. Em texto: entrada = "I HAD always", alvo = "HAD always thought".

Context length

Tamanho do contexto (janela de contexto)

Definição: Número máximo de tokens que uma amostra de entrada contém simultaneamente — e, por extensão, o número máximo de tokens anteriores que o modelo consegue considerar ao prever o próximo.

Função no modelo: Determina a dimensão da sequência de entrada e da tabela de *Positional embedding*; limita quanto contexto passado o modelo pode usar em cada predição.

Relação com outros conceitos: Parâmetro central da *Sliding window*; quanto maior o contexto, menos amostras um corpus de tamanho fixo produz (quando o `stride` acompanha o contexto), mas mais informação cada amostra carrega.

Exemplo: Contexto de 4 tokens gera 1 286 amostras no corpus de teste; contexto de 128 tokens gera apenas 40 amostras a partir do mesmo corpus (ver Seção de Experimentos).

Dataset

Conjunto de dados (`torch.utils.data.Dataset`)

Definição: Estrutura que encapsula a lógica de acesso a um conjunto de amostras de treinamento — no caso deste projeto, a classe `GPTDatasetV1`, que aplica a *Sliding window* sobre o corpus tokenizado e expõe cada par (entrada, alvo) por índice.

Função no modelo: Padroniza a forma como os dados são armazenados e acessados, permitindo que o *DataLoader* os agrupe em lotes de forma genérica, sem conhecer os detalhes do corpus.

Relação com outros conceitos: Implementa a interface exigida pelo *DataLoader* do PyTorch (`__len__` e `__getitem__`); armazena internamente os *Input-target pair* pré-computados.

Exemplo: `GPTDatasetV1(text, tokenizer, max_length=8, stride=8)` produz um objeto com `len(dataset) == 643` amostras para o corpus de teste.

DataLoader

[DataLoader \(carregador de dados\)](#)

Definição: Componente do PyTorch (`torch.utils.data.DataLoader`) que envolve um *Dataset* e organiza suas amostras em lotes (*batches*), podendo embaralhar (*shuffle*) a ordem das amostras a cada época e paralelizar o carregamento.

Função no modelo: É a interface final entre os dados preparados e o laço de treinamento: a cada iteração, entrega um lote de tensores (entrada, alvo) prontos para ser processados pela rede.

Relação com outros conceitos: Consome um *Dataset* (aqui, `GPTDatasetV1`); seu parâmetro `batch_size` define o tamanho do *Batch* (lote), e `drop_last` controla se o último lote incompleto de uma época é descartado.

Exemplo: `create_dataloader_v1(text, batch_size=8, max_length=4, stride=4)` produz lotes com `inputs.shape == (8, 4)`.

Batch

[Lote](#)

Definição: Conjunto de `batch_size` amostras de entrada (e seus respectivos alvos) agrupadas em um único tensor, processadas simultaneamente por uma passagem da rede.

Função no modelo: Permite explorar o paralelismo de hardware (GPU/CPU) e estabiliza a estimativa do gradiente durante o treinamento, ao calcular a perda sobre várias amostras de uma vez em vez de uma única amostra isolada.

Relação com outros conceitos: Produzido pelo *DataLoader*; sua dimensão (`batch_size`) é o primeiro eixo dos tensores de *Token embedding* e *Input embedding*, de forma (`batch_size`, `context_length`, `output_dim`).

Exemplo: Com `batch_size=8` e `max_length=4`, cada lote de entrada tem forma `(8, 4)`; após a camada de embedding com `output_dim=256`, passa a ter forma `(8, 4, 256)`.

4 Embeddings e posição

Embedding

Embedding (representação vetorial densa)

Definição: Vetor de números reais, de dimensão fixa (`output_dim`), que representa um objeto discreto (aqui, um token) em um espaço vetorial contínuo. Ao contrário do Token ID, os valores do vetor são aprendidos durante o treinamento.

Função no modelo: Permite que a rede opere sobre representações contínuas e diferenciáveis, sobre as quais operações como produto interno e multiplicação de matrizes fazem sentido geométrico — o que não é possível com um índice inteiro isolado.

Relação com outros conceitos: Produzido a partir de um *Token ID* por uma *Embedding layer*; a soma do *Token embedding* com o *Positional embedding* resulta no *Input embedding*.

Exemplo: Com `output_dim=256`, cada Token ID é mapeado para um vetor de 256 números reais — por exemplo, o token de ID 40 (“I”) passa a ser representado por um vetor `[0.34, -1.19, ...]` (256 valores) em vez do escalar 40.

Embedding layer

Camada de embedding (`nn.Embedding`)

Definição: Camada de rede neural que funciona como uma tabela de consulta (*lookup table*) de dimensão (`vocab_size`, `output_dim`): dado um Token ID, retorna a linha correspondente da tabela. Os valores da tabela são parâmetros treináveis, inicializados aleatoriamente.

Função no modelo: É a ponte entre o mundo discreto dos *Token ID* e o mundo contínuo em que a rede neural opera. Equivale, matematicamente, a multiplicar um vetor *one-hot* pela matriz de pesos da camada — mas implementada de forma muito mais eficiente, como indexação direta.

Relação com outros conceitos: Instanciada com `vocab_size` igual ao tamanho do *Vocabulary* do tokenizador utilizado; sua saída, somada à saída de uma segunda `nn.Embedding` indexada por posição, forma o *Input embedding*.

Exemplo: `nn.Embedding(50257, 256)` cria uma tabela com $50\,257 \times 256 = 12\,865\,792$ parâmetros treináveis — um por célula da tabela.

Positional embedding

Embedding posicional

Definição: Vetor, de mesma dimensão do *Token embedding*, que codifica exclusivamente a posição de um token dentro da sequência (0, 1, 2, ...), e não seu conteúdo. Neste projeto, segue a abordagem do GPT: posicionamento absoluto e aprendido, via uma segunda `nn.Embedding` indexada pela posição.

Função no modelo: Resolve uma limitação fundamental da camada de embedding pura: como ela é uma tabela de consulta por Token ID, o mesmo token gera sempre o mesmo vetor, não importa em que posição da sequência apareça. Sem informação posicional, o modelo enxergaria a sequência como um conjunto (*bag of tokens*), não como uma sequência ordenada.

Relação com outros conceitos: Somado ao *Token embedding* para formar o *Input embedding*; sua tabela tem tamanho (`context_length`, `output_dim`), diferente da tabela de token embeddings, que tem tamanho (`vocab_size`, `output_dim`).

Exemplo: O token de ID 40 na posição 0 e o mesmo token de ID 40 na posição 2 recebem, após a soma posicional, vetores diferentes — verificado experimentalmente em `src/embeddings.py` (`vetor_pos0 != vetor_pos2`).

Input embedding

Embedding de entrada (token + posição)

Definição: Soma elemento a elemento entre o *Token embedding* de cada posição e o *Positional embedding* correspondente àquela posição. É o vetor final que representa cada token da sequência ao entrar nos blocos do Transformer.

Função no modelo: É literalmente a entrada da arquitetura GPT — o ponto de saída desta Sprint e o ponto de entrada da Sprint seguinte (mecanismo de atenção). Carrega simultaneamente informação de conteúdo (o que o token é) e de posição (onde ele está na sequência).

Relação com outros conceitos: Resultado de *Token embedding* + *Positional embedding*; tem forma `(batch_size, context_length, output_dim)`, a mesma forma esperada pela primeira camada de *self-attention* (Capítulo 3).

Exemplo: Para um lote com `batch_size=8`, `context_length=4` e `output_dim=256`, o *input embedding* tem forma `(8, 4, 256)`.